

# Multi-label Classification (scored task: 4 points)

Tasks (Lab 14):

## Introduction

**Multi-label classification** is a classification problem in which each observation may belong to several classes simultaneously. For an observation with feature vector  $x$ , the response is a binary vector

$$Y = (Y_1, \dots, Y_K) \in \{0, 1\}^K.$$

This differs from ordinary multi-class classification, where each observation belongs to exactly one class.

**Example:** Binary classification/Multi-class classification/Multi-label classification:



- **Binary classification:** Is there a mountain in the image? yes/no.
- **Multi-class classification:** Which single object is shown: mountain, desert, or car?
- **Multi-label classification:** Which labels are present: mountain, trees, forest, lake, sea, glacier? **Answer:**  $Y = (1, 1, 1, 0, 0, 1)$ .

Other examples include medical diagnosis, music tagging, document classification, and recommendation systems.

## Data and starting code

```
import numpy as np

from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression
from sklearn.multiclass import OneVsRestClassifier
from sklearn.multioutput import ClassifierChain

from sklearn.metrics import accuracy_score
from sklearn.metrics import hamming_loss
from sklearn.metrics import jaccard_score

X, Y = fetch_openml("emotions", version=4, return_X_y=True)
Y = (Y == "TRUE").astype(int)

X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.3, random_state=42
)

Technical note. Check your version of scikit-learn. Up to version 1.6.1, ClassifierChain
uses the argument base_estimator. In newer versions, this argument is replaced by estimator.

import sklearn
print(sklearn.__version__)
```

## Task 1: Preliminary analysis of labels

Before fitting models, analyze the label matrix  $Y$ .

1. How many labels are there?
2. What is the empirical frequency of each label?
3. How many active labels does one observation have on average?
4. Which pairs of labels are most strongly correlated?
5. Are the labels approximately independent? Why does this matter for multi-label classification?

## Task 2: Standard multi-label methods

Compare the following methods using logistic regression as the base classifier.

### 1. Binary Relevance (BR)

Binary Relevance fits one binary classifier per label:

$$Y_k \leftarrow X, \quad k = 1, \dots, K.$$

You may use

```
OneVsRestClassifier(LogisticRegression(solver="liblinear")).
```

## 2. Classifier Chains (CC)

Classifier Chains fit binary classifiers sequentially. For a chosen label order,

$$Y_1 \leftarrow X, \quad Y_2 \leftarrow X, Y_1, \quad \dots, \quad Y_K \leftarrow X, Y_1, \dots, Y_{K-1}.$$

At prediction time, previously predicted labels are used as additional input variables for later classifiers.

```
ClassifierChain(order="random", base_estimator=LogisticRegression(solver="liblinear")).
```

Test at least 10 different random label orders.

## 3. Ensemble of Classifier Chains (ECC)

An Ensemble of Classifier Chains fits several classifier chains with different random label orders and combines their predictions. Use at least  $M = 20$  chains and combine predictions by majority voting:

$$\hat{Y}_{ik}^{ECC} = \mathbb{1} \left( \frac{1}{M} \sum_{m=1}^M \hat{Y}_{ik}^{(m)} \geq 0.5 \right).$$

## Evaluation metrics

Let  $Y^{(i)} = (Y_1^{(i)}, \dots, Y_K^{(i)})$  be the true label vector and let  $\hat{Y}^{(i)} = (\hat{Y}_1^{(i)}, \dots, \hat{Y}_K^{(i)})$  be the predicted label vector.

Use the following metrics:

$$\text{SubsetAccuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{1} \left\{ Y^{(i)} = \hat{Y}^{(i)} \right\},$$

$$\text{HammingScore} = \frac{1}{NK} \sum_{i=1}^N \sum_{k=1}^K \mathbb{1} \left\{ Y_k^{(i)} = \hat{Y}_k^{(i)} \right\},$$

$$\text{JaccardScore} = \frac{1}{N} \sum_{i=1}^N \frac{\left| \{k : Y_k^{(i)} = 1\} \cap \{k : \hat{Y}_k^{(i)} = 1\} \right|}{\left| \{k : Y_k^{(i)} = 1\} \cup \{k : \hat{Y}_k^{(i)} = 1\} \right|}.$$

Subset accuracy is strict, Hamming score is label-wise, and Jaccard score focuses on active labels.

```
def evaluate(name, Y_true, Y_pred):
    subset_acc = accuracy_score(Y_true, Y_pred)
    hamming = 1 - hamming_loss(Y_true, Y_pred)
    jacc = jaccard_score(Y_true, Y_pred, average="samples")
    print(
        f"{name:5} {subset_acc:.3f} {hamming:.3f} {jacc:.3f}"
    )
```

**Important.** Jaccard score treats label 1 as the active label. If the meaning of 0 and 1 were swapped, the Jaccard score would change. In contrast, subset accuracy and Hamming score are symmetric with respect to swapping 0 and 1.

## Task 3: Circular Chain Classifier

Implement your own method inspired by the **Circular Chain Classifier (CCC)**.

For each label  $Y_k$ , fit a binary classifier using both the feature vector  $X$  and all remaining labels  $Y_{-k}$ :

$$\hat{p}_k(x, y_{-k}) \approx P(Y_k = 1 \mid X = x, Y_{-k} = y_{-k}).$$

Equivalently, for every  $k = 1, \dots, K$ , fit

$$Y_k \leftarrow X, Y_1, \dots, Y_{k-1}, Y_{k+1}, \dots, Y_K.$$

During training, the true labels are known. During prediction, the true labels are not known and must be estimated iteratively.

### Training

For each label  $k$ :

1. remove column  $Y_k$  from the label matrix;
2. append the remaining label columns  $Y_{-k}$  to the feature matrix  $X$ ;
3. fit a binary classifier predicting  $Y_k$  from  $(X, Y_{-k})$ .

Use logistic regression as the base classifier.

### Prediction

For a new observation, initialize a label vector

$$\hat{y}^{(0)} \in \{0, 1\}^K.$$

Try three initializations: random initialization, all zeros, and BR initialization.

Then choose a permutation of labels  $\sigma = (\sigma(1), \dots, \sigma(K))$  and update labels sequentially:

$$\hat{y}_{\sigma(j)} \leftarrow \mathbb{1}(\hat{p}_{\sigma(j)}(x, \hat{y}_{-\sigma(j)}) \geq 0.5), \quad j = 1, \dots, K.$$

Repeat full passes through all labels until no label changes after a full pass. To avoid infinite loops, set for example

$$\text{max\_iter} = 20.$$

The procedure is heuristic. It may depend on initialization and update order. If stabilization is not reached within `max_iter`, return the last iterate and report that convergence was not achieved.

### Experiments

1. Implement functions `fit_ccc` and `predict_ccc`.
2. Compare CCC with random initialization, zero initialization, and BR initialization.
3. Check whether the result depends strongly on the random initialization.
4. Check whether the result depends strongly on the permutation  $\sigma$ .
5. Compare CCC with BR, CC, and ECC using the same evaluation metrics.
6. Report the average number of iterations needed until stabilization.

## Questions

Answer the following questions briefly.

1. What label-dependence assumption is made by Binary Relevance?
2. Why can the order of labels influence Classifier Chains?
3. Why should ECC be more stable than a single Classifier Chain?
4. Why is subset accuracy usually much smaller than Hamming score?
5. Why is Jaccard score often more informative than Hamming score when active labels are rare?
6. Why can CCC be viewed as searching for a self-consistent label vector?
7. Does CCC define one coherent joint model

$$P(Y_1, \dots, Y_K \mid X = x),$$

or only a collection of conditional models? Why is this distinction important?